Assignment 4: Coherent Multicore System Design

Computer Science and Engineering Department, IIT Madras
Due: 28th October

1 Introduction

In this assignment, you will build and evaluate a shared-memory multicore system using your PA3 processor. You will implement routing and arbitration in a 2D mesh, directory-based MSI coherence, TSO memory ordering, and memory-controller scheduling. You will also implement event counters to measure and check these mechanisms.

The starter supplies the multicore adapter, mesh topology, timed links, finite buffers, packet movement, cache storage, protocol interfaces, SC ordering, store-buffer storage, shared LLC storage and timing, and a DRAM model. Complete the marked functions. The data path is **core → private caches → mesh → home directory and shared LLC → memory controller → DRAM**. The shared LLC is banked across home tiles. The default is a 3×3 mesh with four core tiles, four home/LLC-bank tiles, and one memory-controller tile connected to DRAM. Keep mesh dimensions configurable. The coherent data path has one private cache per core; the migrated PA3 instruction-cache hierarchy remains active.

2 Assignment Resources

2.1 Getting Started

Create `submission4` alongside your previous submissions. Manually migrate your completed PA3 core, execution units, caches, predictor, and speculation logic into the matching starter paths. Follow `README.md`, including its core-interface changes. Do not overwrite A4's framework files or modify previous submissions.

From `submission4`, build and check the supplied framework:

```
make -j4
```

```
make framework-test
```

After migration, check the processor using:

```
./pa4 cpu configs/default.cfg configs/pa3.json
  workloads/arithmetic_programs.txt arithmetic 2000
```

Write this command on one line. Register 3 must equal 12 on cores 0 and 1.

2.2 Implementation Interfaces

Implement `noc/routing.cc`, `coherence/cache.cc`, `coherence/directory.cc`, `memory/ordering.cc`, `memory/scheduling.cc`, and `statistics.cc`. Read `README.md`, `docs/INTERFACES.md`, and `docs/COUNTERS.md`. You may add helper functions and transient-state fields. Preserve public interfaces, capacities, timing, and the documented cycle rules. Run the public tests to check your implementation. Tests for functions you have not completed will fail.

2.3 Memory-Ordering References

For SC, read Leslie Lamport's *How to Make a Multiprocessor Computer That Correctly Executes Multiprocess Programs* (1979). For store buffers, forwarding, and fences, read Sewell et al., *x86-TSO: A Rigorous and Usable Programmer's Model for x86 Multiprocessors* (2010), especially its abstract-machine explanation. Implement the rules in Task 3; implementing the full x86 instruction set or memory types is outside this assignment.

3 Tasks

3.1 Warm Up: Validate Your Migrated Processor

Run the arithmetic example and the framework tests before implementing A4. Confirm that your PA3 prediction, recovery, hazards, and retirement remain functional.

Minimum deliverables. A migrated processor that builds and produces the expected arithmetic register results.

3.2 Task 1: 2D Mesh Routing and Arbitration (20 marks)

Implement XY routing and YX routing. Select one order globally per run. Implement round-robin output arbitration and one policy of your choice, selected by `arbitration custom`. Respect connected ports, bandwidth, finite buffers, and backpressure. All remote communication must traverse the mesh.

Minimum deliverables. Use `pa4-noc` with `workloads/network.traffic`. Check routes, hop counts, and latency for unloaded equal-size packets at increasing distances. Compare both routing modes and arbiters on identical traffic. Include contention and full-buffer tests. Greater hop distance must increase unloaded latency when link timings are equal.

3.3 Task 2: Distributed MSI Coherence (40 marks)

Implement the private-cache and home-directory controllers. Handle read/write misses, upgrades, invalidations, acknowledgements, recalls, clean/dirty eviction, and atomic exchange. Track stable/transient states, owners, sharers, and active transactions. Integrate the shared LLC with coherence, including fills, dirty-data handling, and eviction.

Permit one active cache access per core and one directory transaction per line. Queue conflicting requests and service protocol messages while a core is stalled. Obtain modified data from its owner through the home. Grant write permission only after required invalidation acknowledgements. Preserve dirty data. Apply a pending access before acknowledging its grant. Atomic exchange returns the old word and updates it indivisibly.

Minimum deliverables. Test local hits, remote reads of modified data, multiple readers followed by a writer, competing writers, dirty eviction, false sharing versus separate lines, and a lock-protected counter. Check architectural results and coherence traffic. No writable copy may coexist with another readable copy of the same line.

3.4 Task 3: Memory Ordering (15 marks)

Retain Sequential Consistency (SC): memory accesses appear in one global order that preserves each core's program order. Implement Total Store Order (TSO) with the following rules. Each core buffers non-speculative stores in a bounded FIFO and drains them in order. A load forwards from the youngest older buffered store to the same word; otherwise it may execute before older stores to different words become visible. Preserve load order; stall stores when the buffer is full. A full `FENCE` waits for older memory accesses and an empty store buffer before allowing younger memory accesses. Atomic `XCHG` has the same ordering and performs an indivisible exchange. Select SC or TSO through configuration; TSO is not an instruction. Halted cores service coherence; terminate only after buffered stores and outstanding work drain.

Minimum deliverables. Check that a load reads its core's latest buffered store to the same word, that a load to a different word can proceed while an older store is buffered, and that a full buffer stalls the next store. Check that `FENCE` holds younger memory accesses until older accesses finish and the store buffer is empty.

Run the following test with $x = y = 0$, placing x and y on different cache lines. Each core executes its column from top to bottom.

Core 0	**Core 1**
$x \leftarrow 1$	$y \leftarrow 1$
$r_0 \leftarrow y$	$r_1 \leftarrow x$

Under SC, r_0 and r_1 must not both be 0. Under TSO, both may be 0 if both stores are still buffered when the loads execute. Insert `FENCE` between the store and load on both cores and repeat: both results must not be 0. A TSO run may also produce other valid results; two zeros are not required on every run.

3.5 Task 4: Memory Scheduling (10 marks)

Implement FCFS and one policy of your choice, selected by `scheduler custom`, using the supplied eligible request indices. Preserve dependencies and DRAM timing. Define tie-breaking and starvation prevention. Do not modify the DRAM model.

Minimum deliverables. Compare FCFS and your policy on same-row, alternating-row, independent-bank, and mixed read/write traffic. Measure completion cycles, request latency, row hits/misses/conflicts, and maximum per-core waiting time. Use identical configurations.

3.6 Task 5: Event Counters (10 marks)

Populate the declared counters for network transfers, cache accesses, coherence messages/transactions, store buffering, forwarding, fences, memory requests, row behavior, occupancy, stalls, and latency. Count accepted events once; retries are not new events. Reset clears measurements. Sample occupancy at cycle start and each stall at most once per resource per cycle.

Minimum deliverables. Populate the supplied counter JSON output. Check: injected = delivered + in-network; transactions started = completed + active; stores enqueued = drained + buffered; memory requests accepted = completed + outstanding. Follow the precise counter definitions in the starter. Counters will be cross-checked with independent observations.

4 Submission and Evaluation

Place the complete source code in `submission4` inside your existing pipesim repository. Preserve the starter's directory structure and leave previous submissions unchanged. Commit and push by the announced deadline. Ensure each group member's contribution is attributable in Git history.

Only source code is required. Do not submit reports, plots, logs, generated counter outputs, or reproduction documents. The task deliverables above describe required implementation behavior and validation; they are not separate submission files.

Evaluation considers functional correctness, protocol/order invariants, progress, counter accuracy, policy performance under fixed resources, and code clarity. Code clarity and integration carry the remaining **5 marks**. Consistent counters alone do not prove correctness. Dropping work or changing timing does not earn performance credit. Each member must be able to explain their component and one complete memory transaction in the viva.